

Introdução a Sistemas Distribuídos

Computação Distribuída

Faculdade de Computação e Informática – FCI
Universidade Presbiteriana Mackenzie

Distribuído versus Descentralizado

O que temos ...



(a) Centralizado



(b) Descentralizado



(c) Distribuído

Quando um sistema descentralizado se torna distribuído?

- Adicionando 1 ligação entre dois nós em um sistema descentralizado?
- Adicionando 2 ligações entre dois outros nós?
- Em geral: adicionando $k > 0$ ligações...?

Abordagem alternativa

Duas visões sobre a realização de sistemas distribuídos"

- **Visão integrativa:** conectar sistemas de computadores em rede existentes em um sistema maior.
- **Visão expansiva:** um sistema de computadores em rede existente é estendido com computadores adicionais.

Duas definições

- Um **sistema descentralizado** é um sistema de computadores em rede no qual processos e recursos estão *necessariamente* distribuídos por vários computadores.
- Um **sistema distribuído** é um sistema de computadores em rede no qual processos e recursos estão *suficientemente* distribuídos por vários computadores.

Alguns equívocos comuns

Soluções centralizadas não escalam

Distinção entre centralizado *logicamente* e *fisicamente*. A raiz do Sistema de Nomes de Domínio (DNS):

- logicamente centralizado
- fisicamente (massivamente) distribuído
- descentralizado entre várias organizações

Soluções centralizadas têm um único ponto de falha

Geralmente não é verdade (por exemplo, a raiz do DNS). Um único ponto de falha é frequentemente:

- mais fácil de gerenciar
- mais fácil de tornar mais robusto

Alguns equívocos comuns (cont.)

! Importante

Existem muitos equívocos mal fundamentados sobre *escalabilidade*, *tolerância a falhas*, *segurança*, etc. Precisamos desenvolver habilidades pelas quais os sistemas distribuídos possam ser prontamente compreendidos para julgar tais equívocos.

Perspectivas sobre sistemas distribuídos

Sistemas distribuídos são complexos: adote perspectivas

- **Arquitetura:** organizações comuns
- **Processo:** que tipo de processos e seus relacionamentos
- **Comunicação:** facilidades para troca de dados
- **Coordenação:** algoritmos independentes de aplicação
- **Nomeação:** como você identifica recursos?
- **Consistência e replicação:** o desempenho requer dados, que precisam ser os *mesmos*
- **Tolerância a falhas:** continue funcionando na presença de falhas parciais
- **Segurança:** garanta o acesso autorizado aos recursos

O que queremos alcançar?

Objetivos gerais de *design*

- Suporte ao compartilhamento de recursos
- Transparência de distribuição
- Abertura
- Escalabilidade

Compartilhando recursos

Exemplos canônicos

- Armazenamento e arquivos compartilhados baseados em nuvem
- Streaming multimídia assistido por peer-to-peer
- Serviços de e-mail compartilhados (pense em sistemas de e-mail terceirizados)
- Hospedagem Web compartilhada (pense em redes de distribuição de conteúdo)

Observação

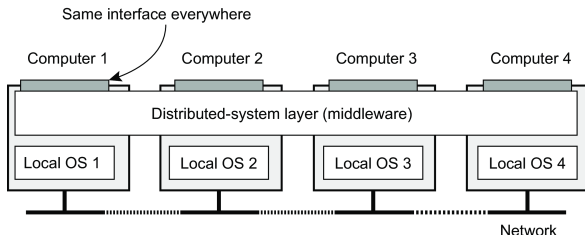
“A rede é o computador”

(citação de John Gage, então na Sun Microsystems)

Transparência de distribuição

O que é transparência?

O fenômeno pelo qual um sistema distribuído tenta *ocultar* o fato de que seus processos e recursos são *fisicamente distribuídos por vários computadores*, possivelmente *separados por grandes distâncias*.



💡 Observação

A transparência de distribuição é tratada por meio de muitas técnicas diferentes em uma camada entre aplicações e sistemas operacionais: uma camada de *middleware*

Tipos de Transparência de distribuição.

Transparência	Descrição
Acesso	Oculta diferenças na representação de dados e como um objeto é acessado.
Localização	Oculta onde um objeto está localizado.
Realocação	Oculta que um objeto pode ser movido para outro local enquanto está em uso.
Migração	Oculta que um objeto pode se mover para outro local.
Replicação	Oculta que um objeto é replicado.
Concorrência	Oculta que um objeto pode ser compartilhado por vários usuários independentes.
Falha	Oculta a falha e a recuperação de um objeto.

Grau de transparência

Visar a transparência total da distribuição pode ser demais

- Há latências de comunicação que não podem ser escondidas.
- **Ocultar completamente as falhas** de redes e nós é (na teoria e na prática) impossível.
 - Você não pode distinguir um computador lento de um com falha.
 - Você nunca pode ter certeza que um servidor realmente realizou uma operação antes de um *crash*.
- Transparência total vai **custar desempenho**, expondo a distribuição do sistema.
- Manter as réplicas *exatamente* atualizadas com o *master* **leva tempo**.
- Descarregar imediatamente as operações de escrita no disco para tolerância a falhas.

Grau de transparência

Expor a distribuição pode ser bom

- Fazer uso de serviços baseados em localização (encontrar amigos próximos)
- Ao lidar com usuários em diferentes fusos horários.
- Quando torna mais fácil para um usuário entender o que está acontecendo (quando e.g., um servidor não responde por um longo tempo, reporte isso como falha).

i Conclusão

Transparência de distribuição é um bom objetivo, mas alcançá-lo é uma história diferente, e muitas vezes nem deveria ser o objetivo.

Abertura de sistemas distribuídos

Sistema distribuído aberto

Um sistema que *oferece componentes* que podem facilmente ser usados por, ou *integrados em outros sistemas*. Um sistema distribuído aberto em si geralmente consistirá de componentes que se originam de outros lugares.

Sobre o que estamos falando?

Ser capaz de interagir com serviços de outros sistemas abertos, independentemente do ambiente subjacente:

- Sistemas devem obedecer a *interfaces* bem definidas.
- Sistemas devem *interoperar* facilmente.
- Sistemas devem suportar a *portabilidade* de aplicações.
- Sistemas devem ser facilmente *extensíveis*.

Políticas versus mecanismos

Implementando abertura: *políticas*

- Qual nível de consistência nós exigimos para dados em cache do cliente?
- Quais operações permitimos que o código baixado realize?
- Quais requisitos de QoS ajustamos em face de largura de banda variável?
- Que nível de sigilo exigimos para a comunicação?

Implementando abertura: *mecanismos*

- Permitir configuração (dinâmica) de políticas de caching.
- Suportar diferentes níveis de confiança para código móvel.
- Fornecer parâmetros de QoS ajustáveis por fluxo de dados.
- Oferecer diferentes algoritmos de criptografia.

Sobre separação estrita

Observação

Quanto mais estrita a separação entre política e mecanismo, mais precisamos garantir mecanismos adequados, potencialmente levando a muitos parâmetros de configuração e gerenciamento complexo.

Encontrando um equilíbrio

Políticas codificadas muitas vezes simplificam o gerenciamento e reduzem a complexidade em troca de menos flexibilidade. Não há uma solução óbvia.

Confiabilidade (*Dependability*)

Conceitos básicos

Um *componente* fornece *serviços* para *clientes*. Para fornecer serviços, o componente pode precisar dos serviços de outros componentes $\Rightarrow$ um componente pode *depend*er de algum outro componente.

Especificamente

Um componente C depende de C^* se a *correção* do comportamento de C depende da correção do comportamento de C^* . (Componentes são processos ou canais.)

Dependability (Confiabilidade)

Requisitos relacionados à confiabilidade

Requisito	Descrição
Disponibilidade	Prontidão para uso
Confiabilidade	Continuidade da entrega do serviço
Segurança	Probabilidade muito baixa de catástrofes
Manutenibilidade	Quão fácil um sistema com falha pode ser reparado

Confiabilidade (*Reliability*) versus disponibilidade

Confiabilidade $R(t)$ do componente C Probabilidade condicional de que C tenha funcionado corretamente durante $[0, t)$ dado que C estava funcionando corretamente no tempo $T = 0$.

Métricas tradicionais

- **Tempo Médio Para Falha (MTTF):** O tempo médio até que um componente falhe.
- **Tempo Médio Para Reparo (MTTR):** O tempo médio necessário para reparar um componente.
- **Tempo Médio Entre Falhas (MTBF):** Simplesmente $MTTF + MTTR$.

Terminologia

Falha, erro, defeito (*Failure, error, fault*)

Termo	Descrição	Exemplo
Falha	Um componente não está cumprindo suas especificações	Programa travado
Erro	Parte de um componente que pode levar a uma falha	Bug de programação
Defeito	Causa de um erro	Programador desleixado

Terminologia

Lidando com defeitos.

Termo	Descrição	Exemplo
Prevenção de defeitos	Previne a ocorrência de um defeito.	Não contrate programadores desleixados
Tolerância a defeitos	Construa um componente e faça com que ele mascare a ocorrência de um defeito.	Construa cada componente por dois programadores independentes.
Remoção de defeitos	Reduz a presença, número, ou severidade de um defeito.	Livre-se de programadores desleixados.
Previsão de defeitos	Estima a presença atual, incidência futura, e consequências dos defeitos.	Estima como um recrutador está indo quando se trata de contratar programadores desleixados.

Sobre segurança

Observação

Um sistema distribuído que não é seguro, não é confiável (*dependable*)

O que precisamos:

- **Confidencialidade:** informação é revelada apenas a partes autorizadas.
- **Integridade:** Assegurar que alterações aos ativos de um sistema só possam ser feitas de forma autorizada.

Autorização, Autenticação, Confiança:

- **Autenticação:** verificar a correção de uma identidade reivindicada.
- **Autorização:** uma entidade identificada tem direitos de acesso adequados?
- **Confiança:** uma entidade pode ter certeza de que outra realizará ações específicas de acordo com uma expectativa específica.

Mecanismos de segurança

Mantendo simples

É tudo sobre *criptografar* e *descriptografar* dados usando *chaves de segurança*.

Notação

$K(\text{dados})$ denota que usamos a chave K para criptografar/descriptografar *dados*.

Criptossistema simétrico

Com chave de criptografia $E_K(\text{data})$ e chave de descriptografia $D_K(\text{data})$:

if $\text{data} = D_K(E_K(\text{data}))$ then $D_K = E_K$.

! Nota

As chaves de criptografia e descriptografia são a mesma e devem ser mantidas *secretas*.

Mecanismos de segurança (cont.)

Criptossistema assimétrico

Chaves distintas: uma chave *pública* $PK(data)$ e uma *privada* (secreta) $SK(data)$.

- Criptografar mensagem de *Alice* para *Bob*: $data = \underbrace{SK_{bob}(\overbrace{PK_{bob}(data)}^{\text{Enviado por Alice}})}_{\text{Ação por Bob}}.$

- Assinar mensagem para *Bob* por *Alice*:

$$[data, \underbrace{data \stackrel{?}{=} PK_{alice}(SK_{alice}(data))}_{\text{Verificar por Bob}}] = [\underbrace{data, SK_{alice}(data)}_{\text{Enviado por Alice}}]$$

Mecanismos de segurança

Hashing seguro

Na prática, usamos *funções hash seguras*: $H(data)$ retorna uma *string* de comprimento fixo:

- Qualquer alteração de $data$ para $data^*$ levará a uma *string* completamente diferente $H(data^*)$.
- Dado um valor hash, é computacionalmente impossível encontrar um $data$ com $h = H(data)$

Assinaturas digitais práticas

Assinar mensagem para *Bob* por *Alice*:

$$[data, \underbrace{H(data) \stackrel{?}{=} PK_{alice}(sgn)}}_{\text{Verificar por Bob}}] = \underbrace{[data, H, sgn = SK_{alice}(H(data))]}_{\text{Enviado por Alice}}.$$

Escala em sistemas distribuídos

! Observação

Muitos desenvolvedores de sistemas distribuídos modernos usam facilmente o adjetivo “escalável” sem deixar claro o *porquê* do seu sistema realmente escalar.

Pelo menos três componentes

- Número de usuários ou processos (*escalabilidade de tamanho*).
- Distância máxima entre nós (*escalabilidade geográfica*).
- Número de domínios administrativos (*escalabilidade administrativa*).

Observação

A maioria dos sistemas considera apenas, até certo ponto, a escalabilidade em tamanho. Frequentemente uma solução: múltiplos servidores poderosos operando independentemente em paralelo. Atualmente, o desafio ainda se encontra na escalabilidade geográfica e administrativa.

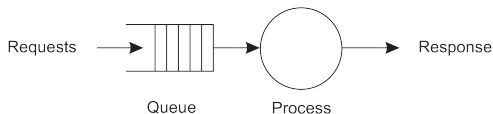
Escalabilidade de tamanho

Causas raízes para problemas de escalabilidade com soluções centralizadas

- A capacidade computacional, limitada pelas CPUs
- A capacidade de armazenamento, incluindo a taxa de transferência entre CPUs e discos
- A rede entre o usuário e o serviço centralizado

Análise formal

Um serviço centralizado pode ser modelado como um sistema de filas simples



Suposições e notações

- A fila tem capacidade infinita $\Rightarrow$ taxa de chegada de requisições não é influenciada pelo tamanho atual da fila ou pelo que está sendo processado.
- Taxa de chegada de requisições: λ
- Capacidade de processamento do serviço: μ requisições por segundo

Fração de tempo tendo k requisições no sistema

$$p_k = \left(1 - \frac{\lambda}{\mu}\right) \left(\frac{\lambda}{\mu}\right)^k$$

Análise formal

Utilização U de um serviço é a fração de tempo em que ele está ocupado

$$U = \sum_{k>0} p_k = 1 - p_0 = \frac{\lambda}{\mu} \Rightarrow p_k = (1 - U)U^k$$

Número médio $\bar{N}$ de requisições no sistema:

$$\bar{N} = \sum_{k \geq 0} k \cdot p_k = \sum_{k \geq 0} k \cdot (1 - U)U^k = (1 - U) \sum_{k \geq 0} k \cdot U^k = \frac{(1 - U)U}{(1 - U)^2} = \frac{U}{1 - U}$$

Vazão média

$$X = \underbrace{U \cdot \mu}_{\text{servidor em trabalho}} + \underbrace{(1 - U) \cdot 0}_{\text{servidor ocioso}} = \frac{\lambda}{\mu} \cdot \mu = \lambda$$

Análise formal

Tempo de resposta: tempo total para processar uma requisição após o envio.

$$R = \frac{\bar{N}}{X} = \frac{S}{1 - U} \Rightarrow \frac{R}{S} = \frac{1}{1 - U}$$

com $S = \frac{1}{\mu}$ sendo o tempo de serviço.

Observações

- Se U é pequeno, tempo de resposta para tempo de serviço é próximo de 1: uma requisição é processada imediatamente.
- Se U vai até 1, o sistema para de funcionar. Solução: diminuir S .

Problemas com escalabilidade geográfica

- Não pode simplesmente ir de LAN para WAN: muitos sistemas distribuídos assumem *interações cliente-servidor síncronas*: cliente envia requisição e espera por uma resposta. *Latência* pode facilmente proibir esse esquema.
- Ligações WAN são frequentemente inerentemente *não confiáveis*: simplesmente mover streaming de vídeo de LAN para WAN está fadado a falhar.
- Falta de *comunicação multiponto*, de modo que uma simples transmissão de busca não pode ser implantada. A solução é desenvolver *serviços de nomeação e diretório* separados (tendo seus próprios problemas de escalabilidade).

Problemas com escalabilidade administrativa

Essência

Políticas conflitantes em relação ao uso (e, portanto, pagamento), gerenciamento e segurança

Exemplos

- *Grids computacionais*: compartilham recursos caros entre diferentes domínios.
- *Equipamento compartilhado*: como controlar, gerenciar e usar um rádio telescópio construído como uma rede de sensores compartilhada em larga escala?

Exceção: várias redes *peer-to-peer*

- Sistemas de compartilhamento de arquivos (baseados, por exemplo, em BitTorrent)
- Telefonia peer-to-peer (versões iniciais do Skype)
- Streaming de áudio assistido por peer (Spotify)

Nota: *usuários finais* colaboram e não *entidades administrativas*.

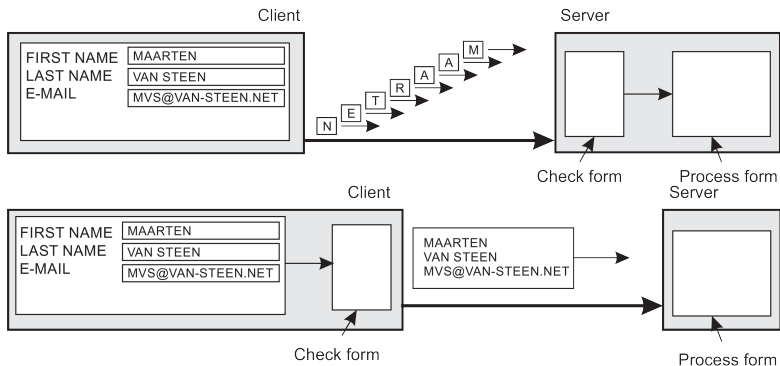
Técnicas para escalar

Ocultar latências de comunicação

- Fazer uso de *comunicação assíncrona*
- Ter manipulador separado para resposta de entrada
- Problema: nem toda aplicação se encaixa nesse modelo

Técnicas para escalar

Facilitar a solução movendo as computações para o cliente.



Técnicas para escalar

Particionar dados e computações entre várias máquinas.

- Mover computações para clientes (Applets e scripts Java)
- Serviços de nomeação descentralizados (DNS).
- Sistemas de informação descentralizados (WWW).

Replicação e *caching*

Fazer cópias de dados disponíveis em diferentes máquinas

- Servidores de arquivos e bancos de dados replicados
- *Websites* espelhados
- *Caches* da Web (em navegadores e *proxies*)
- *Caching* de arquivos (no servidor e cliente)

Escala: o problema com a replicação.

Aplicar replicação é fácil, exceto por uma coisa

- Ter múltiplas cópias (armazenadas em cache ou replicadas), leva a *inconsistências*: modificar uma cópia torna essa cópia diferente do resto.
- Sempre manter as cópias consistentes e de uma forma geral requer *sincronização global* a cada modificação.
- A sincronização global exclui soluções em larga escala.

Observação

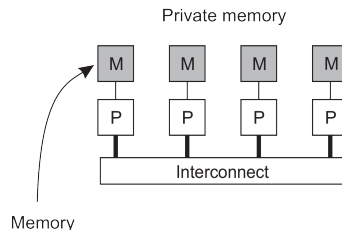
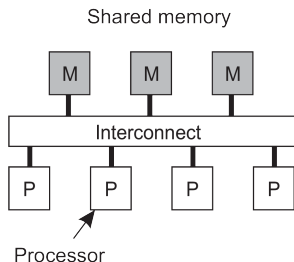
Se pudermos tolerar inconsistências, podemos reduzir a necessidade de sincronização global, mas tolerar inconsistências é *dependente da aplicação*.

Computação paralela

Observação

A computação distribuída de alto desempenho começou com a computação paralela

Multiprocessador e multinúcleo (*multicore*) versus multicomputador



Sistemas de memória compartilhada distribuída

Observação

Multiprocessadores são relativamente fáceis de programar em comparação com multicomputadores, mas têm problemas ao aumentar o número de processadores (ou núcleos). Solução: Tentar implementar um *modelo de memória compartilhada* sobre um multicomputador.

Exemplo através de técnicas de memória virtual

Mapear todas as páginas da memória principal (de diferentes processadores) em um único *virtual espaço de endereçamento*. Se um processo no processador *A* endereça uma página *P* localizada no processador *B*, o SO em *A* *intercepta* e *busca* *P* de *B*, assim como faria se *P* tivesse sido localizado no disco local.

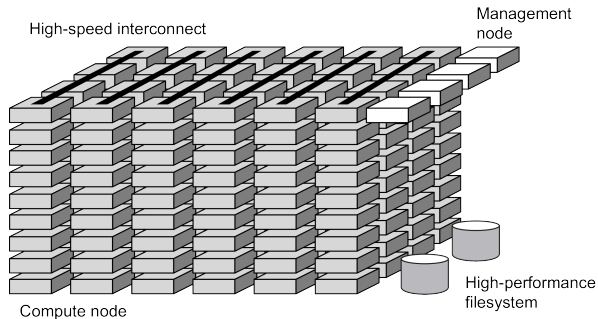
Problema

O desempenho da memória compartilhada distribuída nunca poderia competir com o de multiprocessadores e não atendeu às expectativas dos programadores. Tem sido amplamente abandonado agora.

Computação em cluster

Essencialmente um grupo de sistemas de ponta conectados através de uma LAN

- Homogêneo: mesmo SO, hardware quase idêntico
- Nó(s) de gerenciamento único ou fortemente acoplado



Grid computing

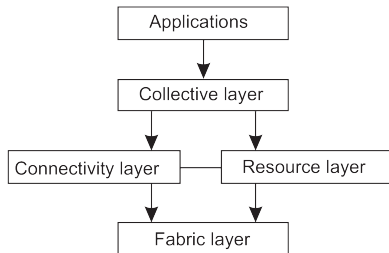
O próximo passo: muitos nós de todos os lugares

- Heterogêneo
- Disperso por várias organizações
- Pode facilmente abranger uma rede de longa distância

Nota

Para permitir colaborações, os *grids* geralmente usam *organizações virtuais*. Em essência, este é um agrupamento de usuários (ou melhor: seus IDs) que permite autorização na alocação de recursos.

Arquitetura para computação em grid



As camadas

- **Fabric:** Fornece interfaces para locais recursos (para consultar estado e capacidades, bloqueio, etc.)
- **Connectivity:** Comunicação/transação protocolos, por exemplo, para mover dados entre recursos. Também vários protocolos de autenticação.
- **Resource:** Gerencia um único recurso, como criar processos ou ler dados.
- **Collective:** Lida com o acesso a múltiplos recursos: descoberta, agendamento, replicação.
- **Application:** Contém as aplicações reais no *grid* em uma única organização.

Integrando aplicações

Situação

Organizações confrontadas com muitas *aplicações em rede*, mas alcançar interoperabilidade era doloroso.

Abordagem básica

Uma aplicação em rede é aquela que roda em um *servidor* tornando seus serviços disponíveis para *clientes* remotos. Integração simples: clientes combinam requisições para (diferentes) aplicações; enviam isso; coletam respostas e apresentam um resultado coerente para o usuário.

Próximo passo

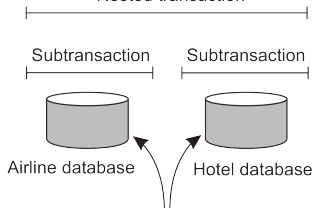
Permitir comunicação direta de aplicação para aplicação, levando a *Enterprise Application Integration*.

Exemplo EAI: transações (aninhadas)

Primitiva	Descrição
BEGIN_TRANSACTION	Marcar o início de uma transação
END_TRANSACTION	Terminar a transação e tentar confirmar (<i>commit</i>)
ABORT_TRANSACTION	Abortar a transação e restaurar os valores antigos
READ	Ler dados de um arquivo, uma tabela ou outro
WRITE	Escrever dados em um arquivo, uma tabela ou outro

Questão: tudo-ou-nada

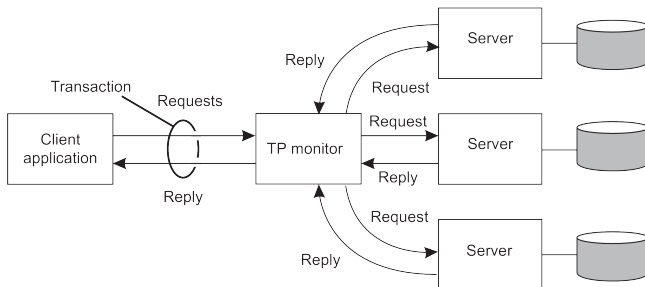
Nested transaction



Two different (independent) databases

- **Atômico:** acontece indivisivelmente (aparentemente)
- **Consistente:** não viola as invariantes do sistema
- **Isolado:** sem interferência mútua
- **Durável:** *commit* significa que as mudanças são permanentes.

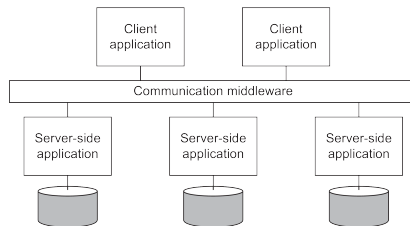
TPM: Transaction Processing Monitor (Monitor de Processamento de Transações)



Observação

Frequentemente, os dados envolvidos em uma transação estão distribuídos por vários servidores. Um *TP Monitor* é responsável por coordenar a execução de uma transação.

Middleware e EAI



Middleware oferece facilidades de comunicação para integração

Remote Procedure Call (RPC): Requisições são enviadas através de chamada de procedimento local, empacotadas como mensagem, processadas, respondidas através de mensagem, e resultado retornado como retorno da chamada.

Message Oriented Middleware (MOM): Mensagens são enviadas para um ponto de contato lógico (publicadas), e encaminhadas para aplicações *subscritas*.

Como integrar aplicações

Transferência de arquivos: Tecnicamente simples, mas não flexível:

- Descobrir formato e layout do arquivo.
- Descobrir gerenciamento de arquivos.
- Propagação de atualizações e notificações de atualização.

Banco de dados compartilhado: Muito mais flexível, mas ainda requer esquema de dados comum, além do risco de gargalo.

Chamada de procedimento remoto: Eficaz quando a execução de uma série de ações é necessária.

Mensageria: RPCs exigem que o chamador e o chamado estejam ativos e funcionando ao mesmo tempo. A mensageria permite o desacoplamento no tempo e no espaço.

Sistemas pervasivos distribuídos

Observação

Próxima geração emergente de sistemas distribuídos nos quais os nós são pequenos, móveis e frequentemente embarcados em um sistema maior, caracterizados pelo fato de que o sistema *naturalmente se mistura ao ambiente do usuário*.

Três subtipos (com sobreposição)

- **Sistemas de computação ubíqua:** pervasivos e *continuamente presentes*, ou seja, há uma interação contínua entre o sistema e o usuário.
- **Sistemas de computação móvel:** pervasivos, mas a ênfase está no fato de que os dispositivos são *inerentemente móveis*.
- Redes de **sensores (e atuadores)**: pervasivas, com ênfase na *detecção* e *atuação* colaborativas do ambiente.

Sistemas ubíquos

Elementos centrais

- ❶ **(Distribuição)** Os dispositivos são conectados em rede, distribuídos e acessíveis de forma transparente
- ❷ **(Interação)** A interação entre usuários e dispositivos é altamente discreta
- ❸ **(Consciência de contexto)** O sistema está ciente do contexto do usuário para otimizar a interação
- ❹ **(Autonomia)** Os dispositivos operam de forma autônoma, sem intervenção humana, e, portanto, são altamente autogerenciados
- ❺ **(Inteligência)** O sistema como um todo pode lidar com uma ampla gama de ações e interações dinâmicas

Computação móvel

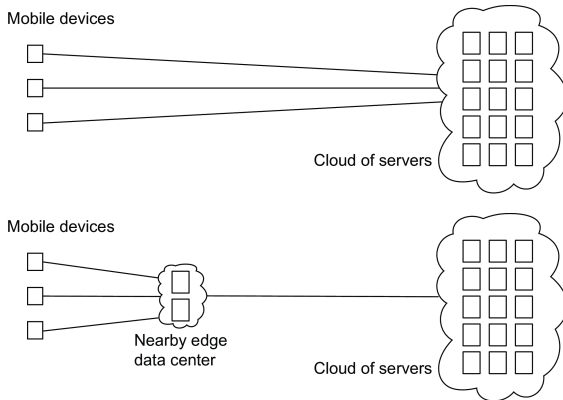
Características distintivas

- Uma miríade de diferentes dispositivos móveis (smartphones, tablets, dispositivos GPS, controles remotos, crachás ativos).
- Móvel implica que a localização de um dispositivo deve mudar ao longo do tempo => mudança de serviços locais, acessibilidade, etc. Palavra-chave: *descoberta*.
- Manter uma comunicação estável pode apresentar sérios problemas.
- Por muito tempo, a pesquisa se concentrou no compartilhamento direto de recursos entre dispositivos móveis. Nunca se tornou popular e agora é considerado um caminho infrutífero para pesquisa.

! Conclusão

Dispositivos móveis estabelecem ligações a servidores estacionários, essencialmente colocando a computação móvel na posição de clientes de serviços baseados na *cloud*.

Computação móvel



Redes de sensores

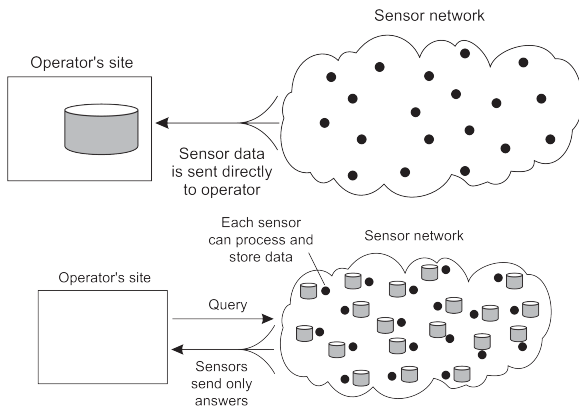
Características

Os *nós* aos quais os sensores estão conectados são:

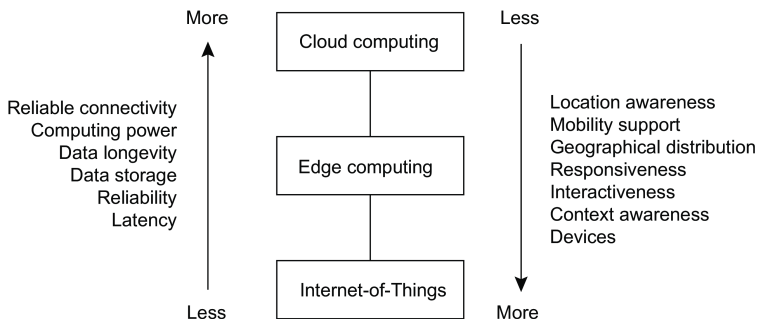
- *Muitos* (10s-1000s)
- *Simples* (pequena memória/capacidade de computação/comunicação)
- Frequentemente alimentados por bateria (ou mesmo sem bateria)

Redes de sensores como bancos de dados distribuídos

Dois extremos



O contínuo nuvem-borda (*cloud-edge continuum*)



Desenvolvendo sistemas distribuídos: Armadilhas

Observação

Muitos sistemas distribuídos são desnecessariamente complexos, causados por erros que exigiram correções posteriores. Muitas *suposições falsas* são frequentemente feitas.

Suposições falsas (e frequentemente ocultas).

- A rede é confiável.
- A rede é segura.
- A rede é homogénea.
- A topologia não se altera.
- Latência é zero.
- Largura de banda é infinita.
- O custo de transporte é zero.
- Há *um* administrador.

That's all Folks!